



«МИРЭА – Российский технологический университет» РТУ МИРЭА

Курсовая работа по дисциплине «Бэкенд-разработка»

Тема работы: «Разработка серверной части веб-приложения для планирования семейного бюджета»

Подготовил: Дицель Н. С.

Группа: ИКБО-11-23

Преподаватель: Волков М.Ю

Актуальность и проблема

Почему это важно

- Контроль доходов и расходов в реальном времени
- Совместный доступ нескольких участников семьи
- Разграничение прав доступа для разных ролей
- Централизованное хранение данных
- Финансовая аналитика и отчётность

Проблемы существующих решений

- Ограниченная поддержка семейных групп
- Слабое разграничение ролей пользователей
- Недостаточная гибкость серверной архитектуры
- Ориентация на индивидуальное использование

Цель разработки: создание серверной части веб-приложения для планирования семейного бюджета на Django с поддержкой REST API, ролевой модели и бюджетными ограничениями.

Сравнительный анализ систем

Возможность	CoinKeeper	Zenmoney	Monefy	Наша система
Учёт доходов и расходов	✓	✓	✓	✓
Семейный бюджет	△	△	✗	✓
Разграничение ролей	△	△	✗	✓
Веб-интерфейс	✓	✓	✗	✓
Управление категориями	✓	✓	✓	✓
Бюджетные ограничения	✓	✓	✗	✓
Импорт/экспорт CSV	✓	✓	✗	✓
REST API	✗	✗	✗	✓
Расширяемость	△	△	✗	✓

Вывод: Разрабатываемая система закрывает недостатки аналогов за счёт семейных групп, ролевой модели, серверной логики, REST API и контроля бюджетных ограничений.

Пользователи и роли системы



Владелец семьи

Создание и управление семейной группой, назначение участников, полный контроль бюджета.



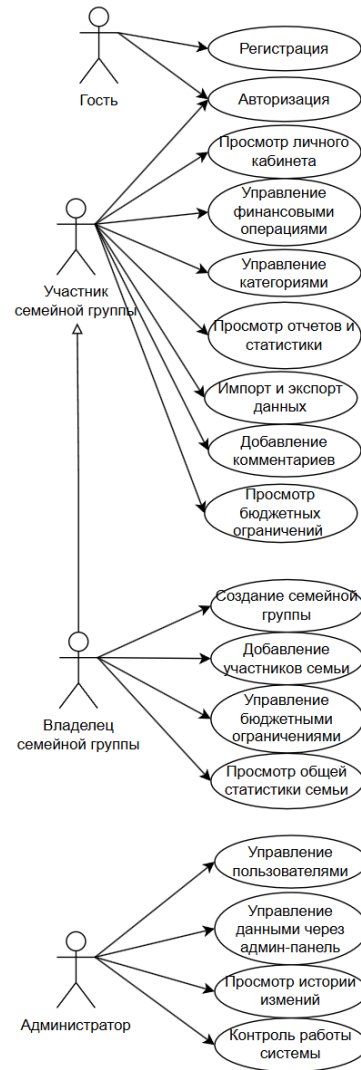
Участник семьи

Добавление транзакций, просмотр бюджета, комментирование операций в рамках своей роли.

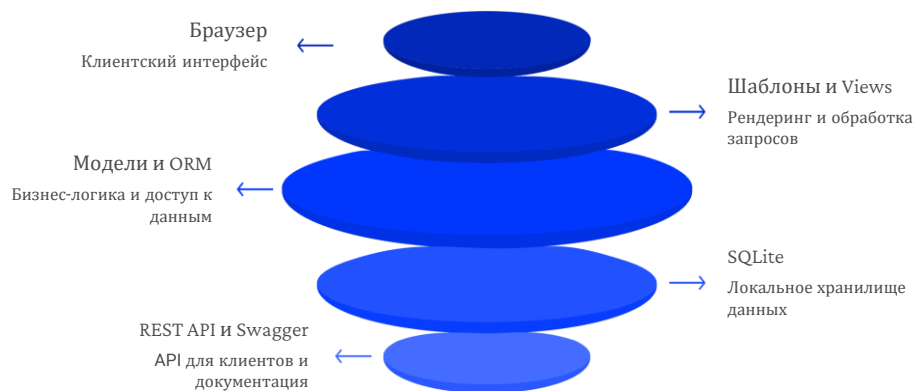


Администратор

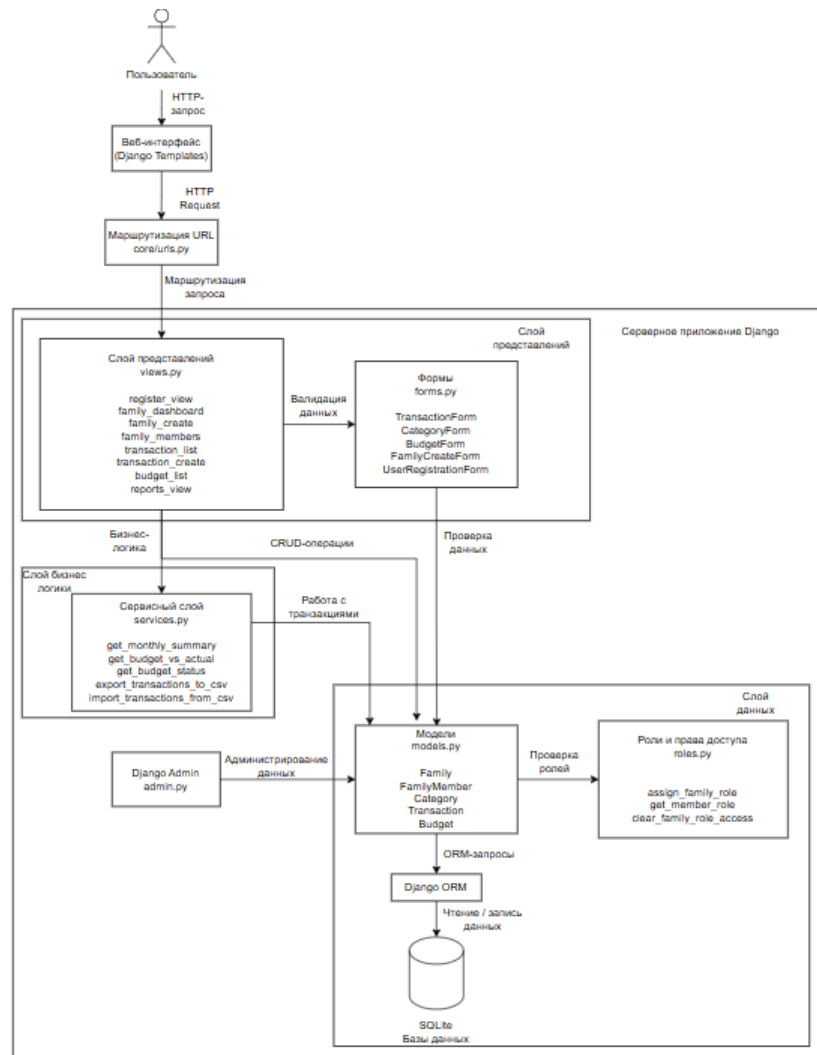
Управление пользователями, мониторинг системы, доступ к полной статистике и отчётам.



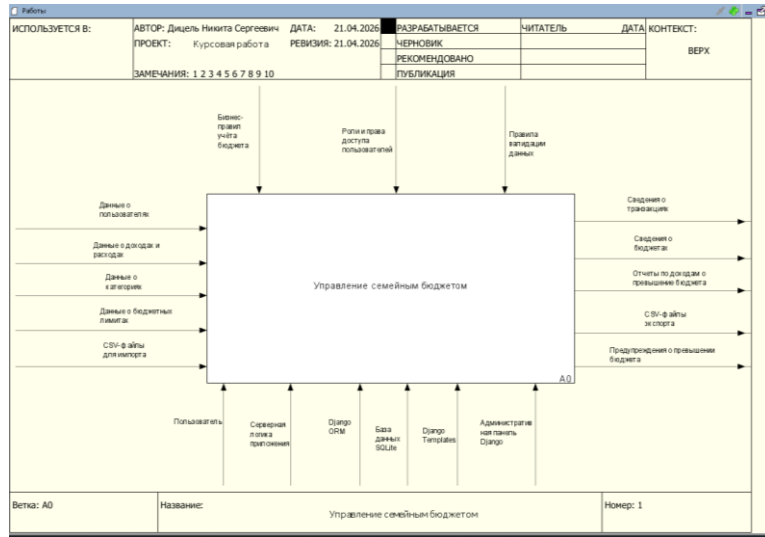
Архитектура веб-приложения



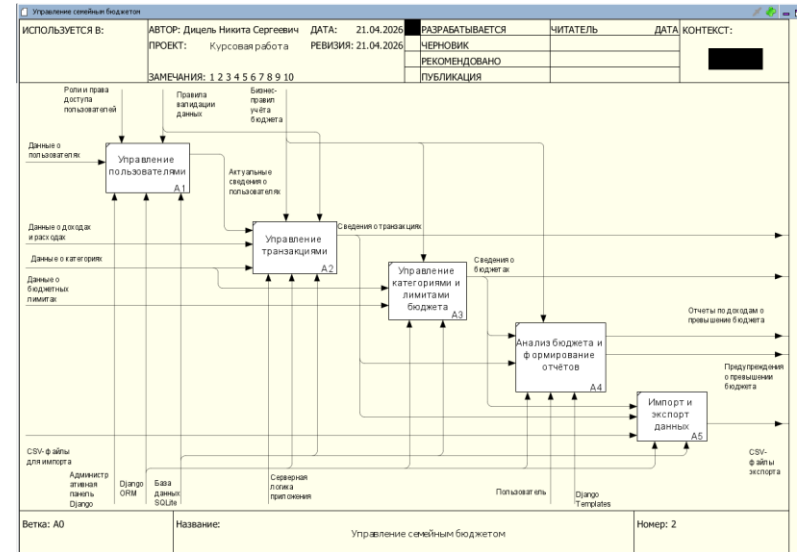
Приложение построено по паттерну MVT (Model-View-Template) с поддержкой REST API для клиентских приложений.



IDEFO0 — процесс управления бюджетом



Процесс «Управление семейным бюджетом»



Декомпозиция процесса «Управление семейным бюджетом»

IDEFO0 помогает формализовать функции: сбор данных → валидация → обработка → отчетность

Используемые технологии



Python

Серверная логика и бизнес-процессы



SQLite3

Реляционная база данных



Django REST Framework

Создание REST API



Git

Управление версиями кода



Django

Веб-фреймворк с MVT архитектурой



Django ORM

Объектно-реляционное отображение



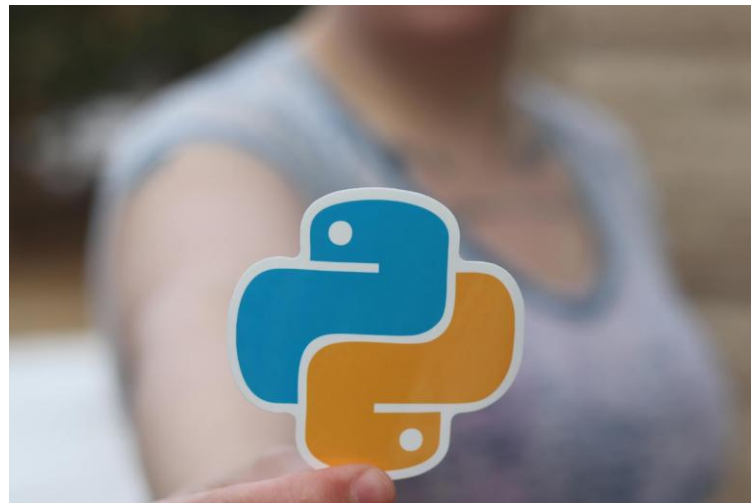
drf-spectacular

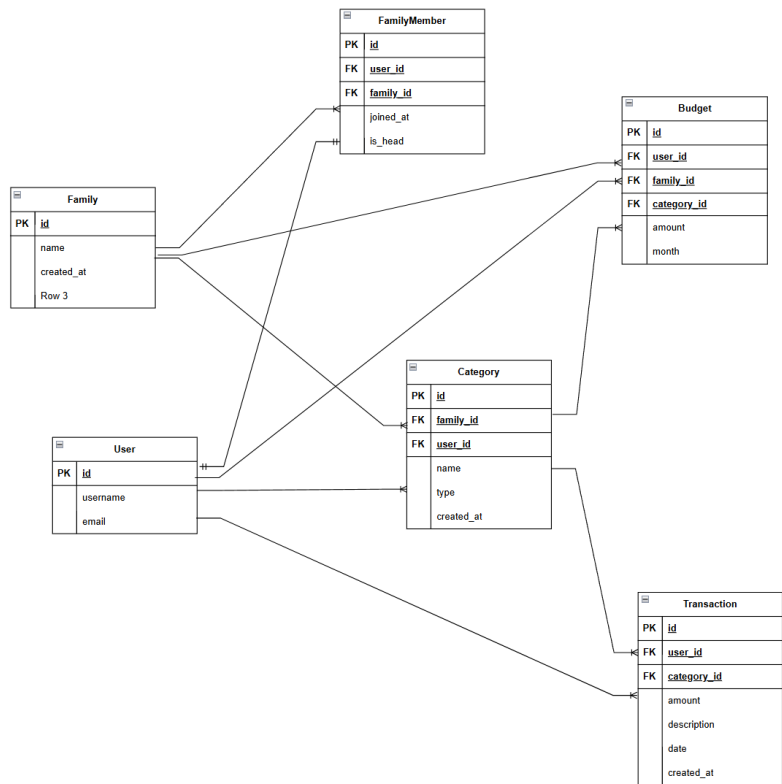
OpenAPI и Swagger UI



VS Code

Среда разработки





Структура базы данных

Основные сущности

- Пользователь (User)
- Семейная группа (Family)
- Участник семьи (FamilyMember)
- Категория (Category)
- Транзакция (Transaction)
- Бюджет (Budget)

Дополнительные сущности

- Комментарий (Comment)
- История изменений (History)
- Права доступа (Permissions)

Связи реализованы через внешние ключи с обеспечением целостности данных.

Реализация серверной логики

01

Авторизация и регистрация

JWT и сессии, хэширование паролей, восстановление доступа

02

Семейные группы

Создание, управление участниками, разграничение прав

03

Категории

Иерархия категорий доходов и расходов

04

Транзакции

CRUD операции, валидация данных, связь с участниками

05

Бюджетные ограничения

Лимиты по категориям и периодам, уведомления о превышении

06

Отчётность и аналитика

Агрегации, фильтрация по периодам, визуализация данных

07

Импорт и экспорт CSV

Массовая загрузка и выгрузка транзакций

08

Комментарии и история

Обсуждение транзакций, аудит изменений

09

REST API

Документированный интерфейс для клиентских приложений

REST API и Swagger UI

Основные endpoint'ы

Модули данных

- /api/families/
- /api/categories/
- /api/transactions/
- /api/budgets/

Служебные модули

- /api/comments/
- /api/history/
- /api/schema/
- /api/docs/

API документируется через Swagger/OpenAPI с автоматической генерацией интерфейса.



Тестирование системы

Модульное тестирование

Тестирование отдельных функций и методов модулей приложения

Интеграционное тестирование

Проверка взаимодействия между компонентами системы

Тестирование REST API

Валидация endpoint'ов через Swagger UI и ручные проверки

Ручное тестирование

Проверка веб-интерфейса, авторизации, транзакций, бюджетов

Проверенные функции

- Авторизация и регистрация
- CRUD транзакций
- Управление категориями
- Бюджетные ограничения
- Отчётность
- Импорт/экспорт CSV

```
PS C:\Users\relax\Documents\kursach_6_backend> python -m pytest tests\ -q
plugins: Faker-40.12.0, cov-4.1.0, django-4.12.0, env-1.6.0, lazy-fixture-0.6.3, pythonpath-0.7.3, xdist-3.8.0
collected 101 items

tests\test_coverage_completeness.py ..... [ 35%]
tests\test_forms.py ..... [ 40%]
tests\test_integration.py .. [ 42%]
tests\test_models.py ..... [ 52%]
tests\test_permissions.py ..... [ 70%]
tests\test_services.py ..... [ 77%]
tests\test_views.py ..... [ 98%]
tests\test_openapi.py .. [100%]

----- coverage: platform win32, python 3.12.10-final-0 -----
Name                                                                    Stmts  Miss  Cover   Missing
-----
family_finance\core\__init__.py                                           0      0  100%
family_finance\core\admin.py                                             35      0  100%
family_finance\core\apps.py                                               4      0  100%
family_finance\core\context_processors.py                                8      0  100%
family_finance\core\forms.py                                             181     0  100%
family_finance\core\migrations\0001_initial.py                           8      0  100%
family_finance\core\migrations\0002_family_alter_budget_unique_together_and_more.py  7      0  100%
family_finance\core\migrations\0003_alter_budget_options_alter_family_options_and_more.py  6      0  100%
family_finance\core\migrations\0004_create_default_groups.py            19      0  100%
family_finance\core\migrations\0005_auto_20260328_1451.py                4      0  100%
family_finance\core\migrations\0006_add_data_integrity_constraints.py     5      0  100%
family_finance\core\migrations\__init__.py                               0      0  100%
family_finance\core\models.py                                           116     0  100%
family_finance\core\openapi.py                                            47      0  100%
family_finance\core\roles.py                                              58      0  100%
family_finance\core\services.py                                          144     0  100%
family_finance\core\urls.py                                               4      0  100%
family_finance\core\views.py                                             362     0  100%
-----
TOTAL                                                                    1008     0  100%

Required test coverage of 100% reached. Total coverage: 100.00%

===== 101 passed in 82.83s (0:01:22) =====
```

Спасибо за внимание!